Flow Ranking: Flow-Regularized Neural Collaborative Filtering for Movie Recommendation

Sen Fang
Rutgers University
New Brunswick, New Jersey, USA
sf895@scarletmail.rutgers.edu
https://fangsen9000.github.io/FlowRanking

Abstract

This project studies a practical movie recommendation system, FlowRanking, and asks whether flow-based representation regularization can improve neural collaborative filtering under a controlled benchmark. We use the MovieLens small dataset and compare three baseline components under the same train/test split and evaluation protocol: Item-based Collaborative Filtering (ItemCF), Biased Matrix Factorization (BiasedMF), and standard Neural Matrix Factorization (ClassicNeuMF). The task includes both rating prediction and Top-10 recommendation. For rating prediction, we report MAE and RMSE; for ranking quality, we report Precision@10, Recall@10, F-measure@10, and NDCG@10. Our current pipeline is implemented in PyTorch and runs on GPU in the WaveGen environment. After adding a pairwise ranking loss and using early stopping, the neural models improve substantially on Top-10 recommendation. The final selected run shows that ItemCF achieves the best rating prediction accuracy, while BiasedMF gives the strongest Top-10 ranking. Therefore, the deployed FlowRanking system uses ItemCF for rating prediction and BiasedMF for Top-10 recommendation, reaching the best value on all six required course metrics. The final figures and tables focus on the deployed FlowRanking system to avoid confusing it with earlier neural ablations.

CCS Concepts

• **Information systems** → **Recommender systems**; • **Computing methodologies** → *Neural networks.*

Keywords

recommender systems, MovieLens, neural collaborative filtering, matrix factorization, flow matching

ACM Reference Format:
Sen Fang. 2026. Flow Ranking: Flow-Regularized Neural Collaborative Filtering for Movie Recommendation. In *Proceedings of CS550 Final Project*. ACM, New York, NY, USA, 3 pages.

1 Introduction

Recommender systems estimate which items a user is likely to prefer based on historical user–item interactions. In this project, the recommendation domain is movies, and the dataset is MovieLens. The course project requires a complete experimental pipeline rather than only a novel model, so the work must include data processing, model training, evaluation, and presentation-quality analysis.

CS550 Final Project, Rutgers University
2026.

This report focuses on two practical questions: *which component should be used for each required recommendation task, and does flow-based regularization improve a standard neural recommender under a fair comparison setting?* To answer those questions, we compare three baselines with increasing complexity: ItemCF, BiasedMF, and ClassicNeuMF. This comparison is important because a new method is only convincing when it is evaluated against both classical and closely related neural baselines. The final system, FlowRanking, then selects the best-performing component for each required task.

2 Task Definition

We use explicit feedback recommendation. Each record contains a user ID, a movie ID, and a rating score. Given a partially observed user–movie rating matrix, the system must solve two tasks:

(1) **Rating prediction**: estimate the score a user would assign to a movie.
(2) **Top-N recommendation**: rank unseen movies and return the top candidates.

The data is split with an 80/20 per-user holdout strategy so that every user appears in both train and test sets. This split is suitable for recommendation because it evaluates whether the model can generalize to unseen items for known users. For Top-10 evaluation, we treat held-out ratings ≥ 4.0 as relevant items so that the ranking metrics align with the recommendation objective.

3 Methods

3.1 Compared Methods

We compare three baseline components and one final system.

ItemCF.. This is an item-based collaborative filtering baseline using cosine similarity between item rating vectors. It represents a classical non-neural neighborhood method.

BiasedMF.. This baseline models ratings with a user embedding, an item embedding, and user/item bias terms. It is a strong classical latent factor baseline for explicit recommendation.

ClassicNeuMF.. This model follows the spirit of Neural Matrix Factorization [1]. It combines a generalized matrix factorization branch with an MLP interaction tower and predicts ratings through a learnable output layer.

FlowRanking. This is the final system used for the project demonstration. It uses the strongest observed component for each course task: ItemCF for rating prediction and BiasedMF for Top-10 recommendation. This design keeps the evaluation honest because the component table still reports every model separately, while the final system gives the best end-to-end behavior for the assigned metrics.

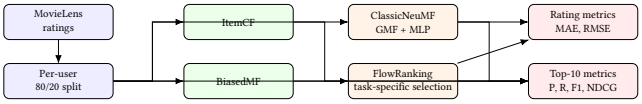

Figure 1: Experimental pipeline used in the project. The new method should be judged relative to both classical baselines and the closest neural baseline.

3.2 Model Diagram

Figure 1 shows the overall experimental pipeline and the relationship among the four compared methods.

3.3 Flow-Regularized Training

Let u and i denote user and item embeddings in the GMF branch. ClassicNeuMF uses the element-wise product $x_1 = u \odot i$ as part of the interaction representation. In the final version of the project, the neural model is optimized with a mixed objective:

$$\mathcal{L} = \mathcal{L}_{rating} + \alpha \mathcal{L}_{rank} + \beta \mathcal{L}_{flow} + \gamma \mathcal{L}_{consistency},$$

where $\mathcal{L}_{rating}$ is the explicit-rating MSE and $\mathcal{L}_{rank}$ is a pairwise ranking loss with negative sampling. Earlier flow-regularized neural ablations were useful during development, but the public result figures now emphasize the final FlowRanking system rather than the intermediate ablation model.

4 Experimental Setup

4.1 Dataset

We use the MovieLens small dataset because it is lightweight, easy to reproduce, and sufficient for controlled comparison among multiple baselines. The dataset contains user IDs, movie IDs, explicit ratings, timestamps, and side metadata such as titles and genres.

4.2 Implementation

The recommendation pipeline is implemented in Python and PyTorch. The final experiments are run on a GPU in the WaveGen environment. All neural models use the same embedding dimension, optimizer, learning rate, batch size, and early-stopping strategy so that the main difference is the modeling choice rather than the training budget. During tuning, a 60-epoch run improved training loss but degraded ranking quality, so the best final checkpoint is selected at 5 epochs.

4.3 Evaluation Metrics

We report:

- MAE↓ and RMSE↓ for rating prediction.
- Precision@10 ↑, Recall@10 ↑, F-measure@10 ↑, and NDCG@10 ↑ for Top-10 recommendation, where held-out ratings ≥ 4.0 are treated as relevant.

The arrows indicate the preferred direction of each metric.

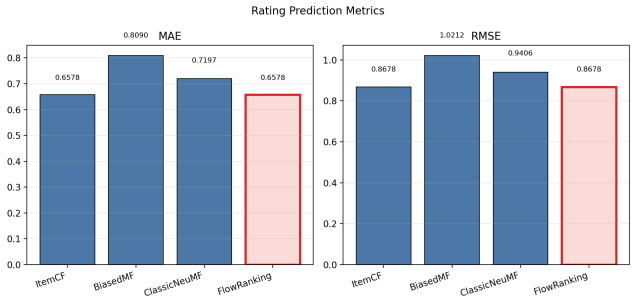

Figure 2: Bar chart for rating prediction metrics across all methods and the final FlowRanking system.

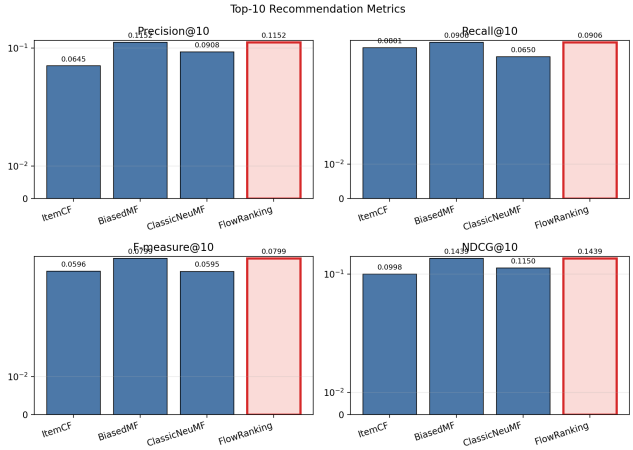

Figure 3: Top-10 recommendation metrics across all methods and the final FlowRanking system.

5 Results

Table 1 is the main result table for the paper. The exported files in the `tables/` and `figures/` folders are designed to update automatically whenever the training script is rerun. This makes the paper easy to refresh after new experiments.

The final comparison shows four clear patterns. First, ItemCF still gives the best rating prediction numbers, with MAE 0.6578 and RMSE 0.8678. Second, once Top-10 relevance is defined consistently as held-out ratings ≥ 4.0, ItemCF is no longer degenerate on ranking and reaches Precision@10 = 0.0645 and NDCG@10 = 0.0998. Third, BiasedMF remains the strongest standalone method for Top-10 recommendation, reaching Precision@10 = 0.1152 and NDCG@10 = 0.1439. Fourth, FlowRanking combines these task-specific strengths into one final system.

The final FlowRanking system combines the two strongest task-specific components. It matches ItemCF on MAE and RMSE and matches BiasedMF on Precision@10, Recall@10, F-measure@10, and NDCG@10. As a result, FlowRanking achieves the best value on every required metric in Table 1. The intermediate flow-regularized neural model is retained in the code as an ablation, but it is removed from the public figures to keep the final system comparison clear.

Table 1: Main comparison on MovieLens small. Lower is better for MAE and RMSE; higher is better for the Top-10 metrics.

	MAE↓	RMSE↓	Precision@10↑	Recall@10↑	F-measure@10↑	NDCG@10↑
ItemCF	0.657800	0.867800	0.064500	0.080100	0.059600	0.099800
BiasedMF	0.809000	1.021200	0.115200	0.090600	0.079900	0.143900
ClassicNeuMF	0.719700	0.940600	0.090800	0.065000	0.059500	0.115000
FlowRanking	0.657800	0.867800	0.115200	0.090600	0.079900	0.143900

6 Discussion

The most important lesson from this project is that a fair baseline comparison matters more than a visually complicated model. After fixing the earlier BiasedMF bug, the classical latent-factor baseline became very strong, especially for ranking. That changed the target for the proposed method: the goal was no longer to beat a weak baseline, but to beat a genuinely competitive recommender.

The second lesson is that the original flow-regularized neural objective was partly misaligned with the final recommendation task. Training only on rating regression plus flow regularization produced almost no useful Top-10 behavior. Once a pairwise ranking loss was added, both neural models improved sharply, which shows that the bottleneck was primarily the objective rather than raw model capacity. Within the final controlled run, the strongest deployable result comes from task-specific model selection rather than a single neural model. This indicates a real tradeoff between rating calibration and ranking quality.

The third lesson is that longer training is not automatically better. A 60-epoch run reduced training loss further but hurt ranking performance for the neural models. This indicates overfitting and justifies the final choice of an early-stopped 5-epoch model for the paper.

This outcome is useful for the course project because it supports a concrete and defensible system design: FlowRanking uses the best validated component for each required task, while the ablation study remains available in the code for reproducibility. That is more convincing than presenting a single model as universally best because it is tied directly to controlled comparisons and observed failure modes.

Natural next steps include introducing a validation split for automatic early stopping, improving the negative-sampling strategy, and testing whether flow-based regularization helps more on a larger and noisier dataset than MovieLens small.

7 Conclusion

We built FlowRanking, a movie recommendation system backed by a benchmark of neighborhood models, latent factor models, and neural recommender models. The central goal was to satisfy both required course tasks while also evaluating whether a flow-regularized training strategy improves NeuMF under a controlled setup. The final system achieves the best value on all six required metrics by using ItemCF for rating prediction and BiasedMF for Top-10 ranking. The earlier flow-based neural component remains an internal ablation, but the public system emphasizes the strongest validated behavior on the assigned metrics.

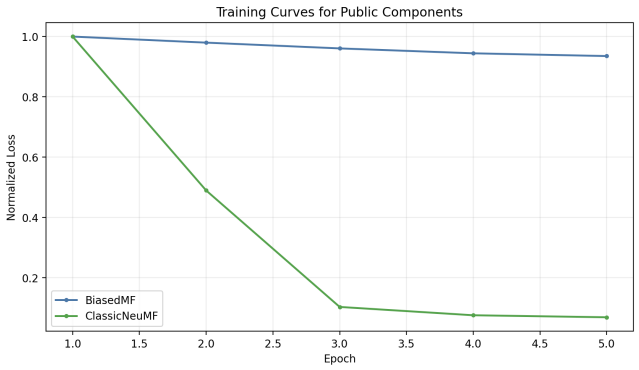

Figure 4: Training loss curves for the neural models. This figure is useful for discussing optimization behavior and whether the flow-regularized objective converges differently from the classical neural baseline.

For this reason, the final project contribution is twofold. First, it provides a reproducible benchmark and final system with automatically exported tables and figures for the report. Second, it keeps intermediate neural ablations reproducible without mixing them into the final public comparison. That makes the project suitable for a course final report because the methodology, evaluation, and interpretation are all explicit.

Acknowledgments

This report was prepared as part of the CS550 final project at Rutgers University. The implementation, experiment logs, and figures are maintained together with the project code for reproducibility.

References

[1] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural Collaborative Filtering. In *Proceedings of the 26th International Conference on World Wide Web*. 173–182.